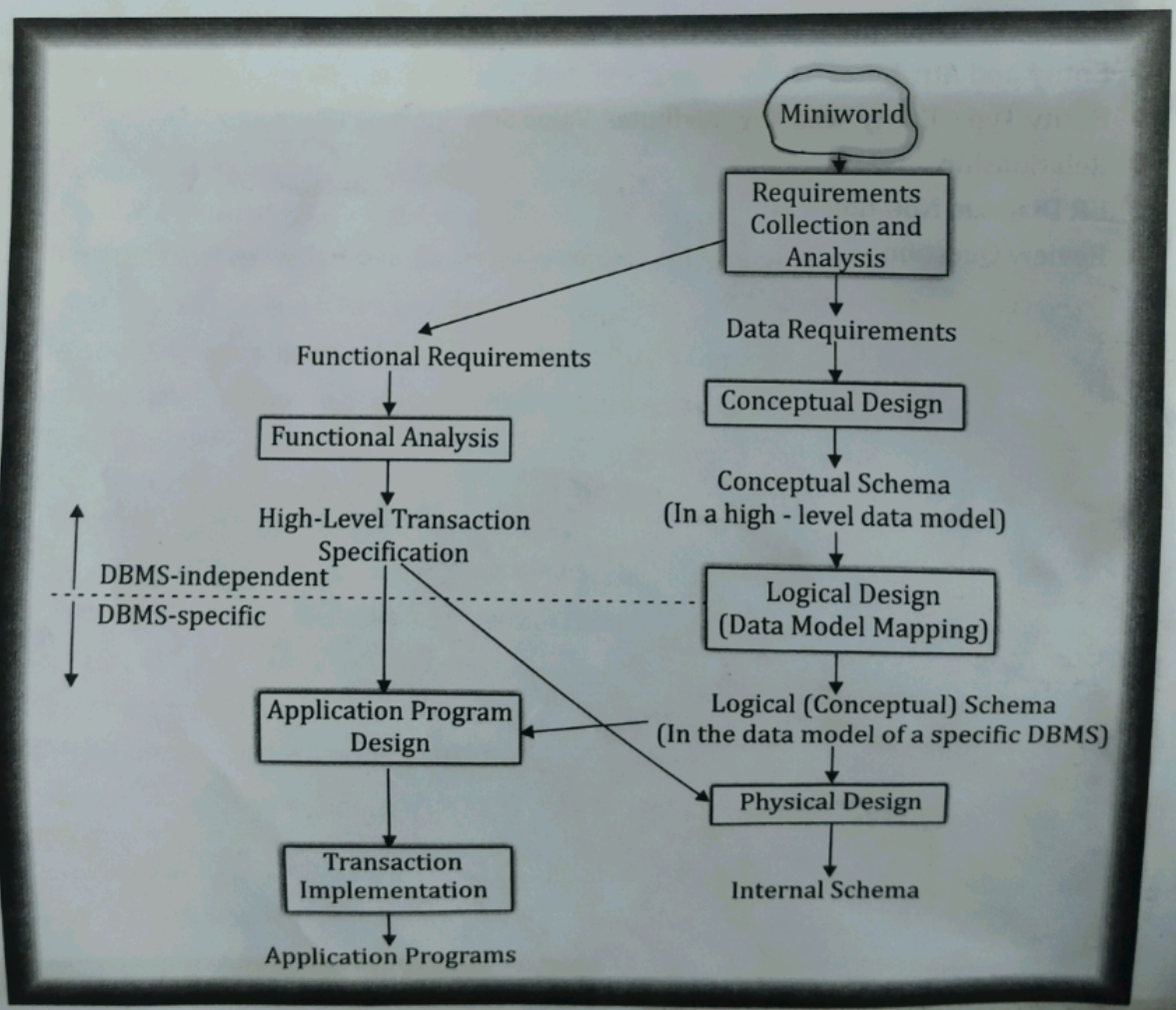
2.1 Introduction

Data modeling is a technique to document a software system using diagrams and symbols. It is used to represent communication of data. The conceptual model is a data model used to describe the conceptual schema. It hides the internal details of physical storage and targets on describing entities, data type, relationships and constraints. Entity-Relationship (ER) model, which is a popular high-level conceptual data model. This model and its variations are frequently used for the conceptual design of database applications, and many database design tools employ its concepts. We describe the basic data structuring concepts and constraints of the ER model and discuss their use in the design of conceptual schemas for database applications. We also present the diagrammatic notation associated with the ER model, known as ER diagrams.

2.2 High Level Conceptual Data Models for Database Design

Figure below shows a simplified description of the database design process.



Phases of Database Design

The first step shown is **requirements collection and analysis**. During this step, the database designers interview prospective database users to understand and document their data requirements. The result of this step is a written set of users' requirements. These requirements should be specified in as detailed and complete a form as possible. In parallel with specifying the data requirements, it is useful to specify the known functional requirements of the application. These consist of the user-defined operations or transactions that will be applied to the database, including both retrievals and updates.

Once all the requirements have been collected and analyzed, the next step is to create a conceptual schema for the database, using a high-level conceptual data model. This step is called **conceptual design**. The conceptual schema is a concise description of the data requirements of the users and includes detailed descriptions of the entity types, relationships, and constraints;these are expressed using the concepts provided by the high-level data model. The high-level conceptual schema can also be used as a reference to ensure that all users' data requirements are met. The functional analysis helps to confirm that the conceptual schema meets all the identified functional requirements.

The conceptual schema is transformed from the high-level data model into the implementation data model. This step is called **logical design** or **data model mapping**. The last step is the **physical design** phase, during which the internal storage structures, indexes, access paths, file organizations and physical for the database files are specified. In parallel with these activities, application programs are designed and implemented as database transactions corresponding to the high-level transaction specifications.

2.3 Entity and Attributes

2.3.1 Entity

Entities are the principal data object about which information is to be collected. Entities are usually recognizable concepts, either concrete or abstract, such as person, places, things (or) events which have relevance to the database. Some specific examples of entities are ***EMPLOYEES, PROJECTS and INVOICES*** etc.

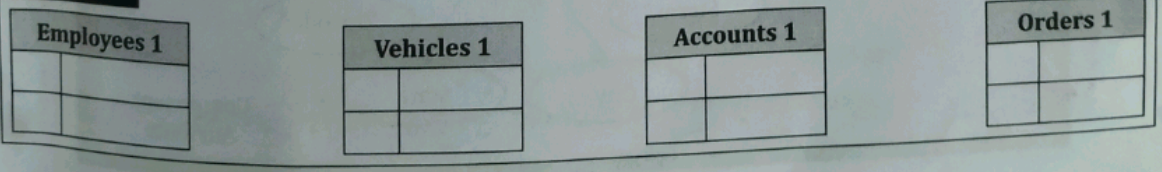
Definition: Entity

An entity may be any object, class, person or place. (or) Entity is an object in the real world with an independent existence and can be differentiated with other object.

An entity might be

- Object with physical existence. Example: lecturer, student, car
- Object with conceptual existence. Example: course, job, position

Example



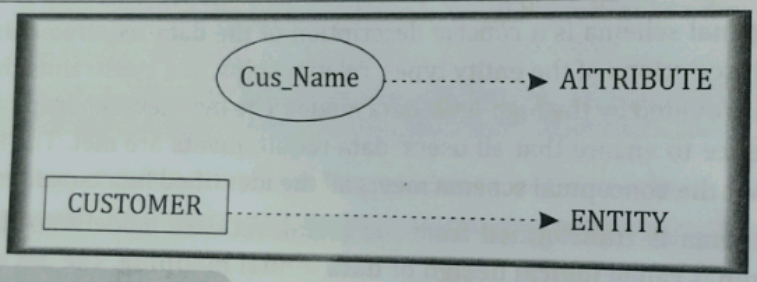
2.3.2 Attributes

The attribute is used to describe the property of an entity. Attributes are facts or description of entities. They are also often nouns and become the columns of the table. For example, for entity student, the attributes can be first name, last name, email, address and phone numbers.

Definition: Attribute

An attribute is a property or description of an entity.

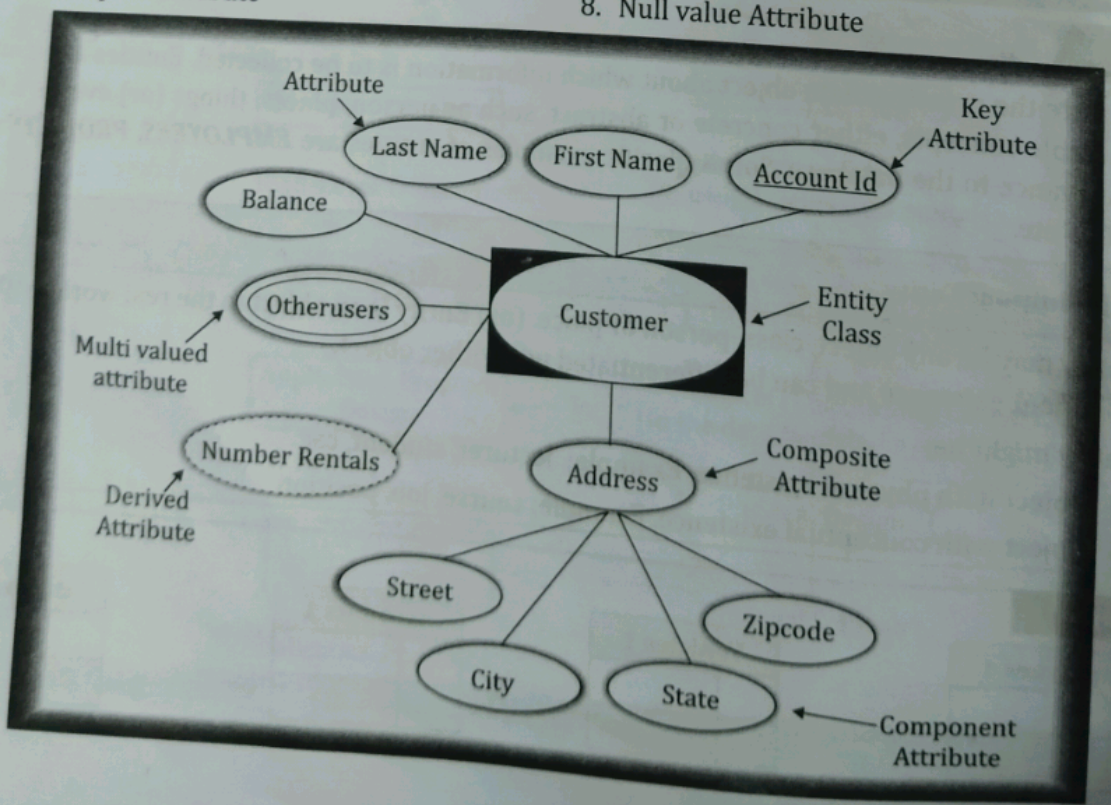
Example: Attributes of the "employee" class might include name, age, address, salary etc.



2.3.3 Types of Attributes

There are different types of Attributes are:

1. Simple Attribute
2. Composite Attribute
3. Single valued Attribute
4. Multi valued Attribute
5. Stored Attribute
6. Derived Attribute
7. Complex Attribute
8. Null value Attribute



1. **Simple Attribute :** Simple attribute that consist of a single atomic value. A simple attribute cannot be subdivided. ***For example*** the attributes age, sex etc.
2. **Composite Attribute :** A composite attribute is an attribute that can be further subdivided and attribute is value not atomic.

   For example the attribute ADDRESS can be subdivided into street, city, state and zip code.
3. **Single-valued Attribute :** A single valued attribute can have only a single value.

   For example a person can have only one 'date of birth', 'age' etc. A single valued attributes can have only single value. But it can be simple or composite attribute. Date of birth is a composite attribute; 'age' is a simple attribute. But both are single valued attributes.
4. **Multi-valued Attribute :** Multivalued attributes can have multiple values.

   For example a person may have multiple phone numbers, multiple degrees etc. Multivalued attributes are shown by a double line connecting to the entity in the ER diagram.
5. **Stored Attribute :** Stored attribute that supplies a value to the related attribute.

   For example 'Date of birth' of a person is a stored attribute.
6. **Derived Attribute :** The value for the derived attribute is derived from the stored attribute.

   For example 'Date of birth' of a person is a stored attribute. The value for the attribute 'AGE' can be derived by subtracting the 'Date of Birth'(DOB) from the current date.
7. **Complex Attribute :** A complex attribute that is both composite and multi valued.

   For example, if a person can have more than one residence and each residence can have multiple phones, an attribute AddressPhone for a PERSON entity type can be specified as shown below.

   {AddressPhone({Phone(AreaCode,PhoneNumber)},Address(StreetAddress(Number,Street,AppartmentNumber),City,State,Zip))}
8. **Null Value Attribute :** A particular entity may not have an applicable value for an attribute.

   For example, the apartment_number attribute of an address applies only to addresses that are in apartment buildings and not to other types of residences, such as single-family homes.

2.4 Entity Type, Entity Sets, Key Attributes, Value Sets

2.4.1 Entity Type

An Entity Type defines a collection of similar entities .An ***entity type*** is a collection of entity instances sharing similar properties and relationships associate from one entity type to another entity type. Entity types are named after the entities that belong to the set of interest. It is a common convention that the names of Entity Types are singular and the first letter is capitalized.

Example

- An entity type named ***Student*** defines a ***collection of student entities***.
- An entity type named ***Invoice*** defines a ***collection of invoice entities***.

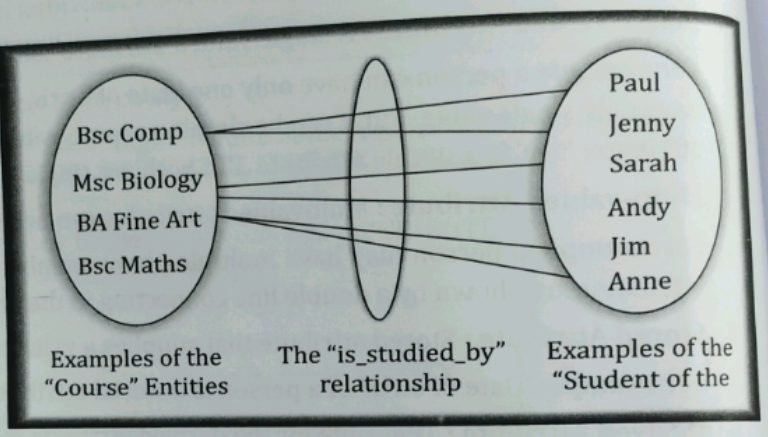
Definition: Entity Type

An Entity Type defines a collection (or) set of entities that have the same attributes.

2.4.2 Entity Sets

An Entity set is collection (or) set all entities of a particular entity type at any point of time.

Example: A company have many employees and these employees are defined as entities (e1,e2,e3....) and all these entities having same attributes are defined under entity type employee and set {e1,e2,.....} is called entity set.



Definition: Entity Set

An Entity Set is a collection of related entities all of the same type. An Entity Set is also known as the extension of an entity type.

2.4.3 Types of Entity

An entity set may be of the following two types- **Strong and Weak Entity.**

2.4.3.1 Strong Entity

A strong entity set is an entity set that contains sufficient attributes to uniquely identify all its entities. In other words, a primary key exists for a strong entity set. Primary key of a strong entity set is represented by underlining it.

Example

Universities and Colleges assign unique student numbers, one per student. Student has a key attribute and would be considered a strong entity type.

Definition: Strong Entity

Entity types that have primary keys are called strong entity type.

2.4.3.2 Weak Entity

A weak entity set is an entity set that does not contain sufficient attributes to uniquely identify its entities. In other words, a primary key does not exist for a weak entity set. An entity that depends on another entity called a weak entity. However, it contains a partial key called as a **discriminator.**

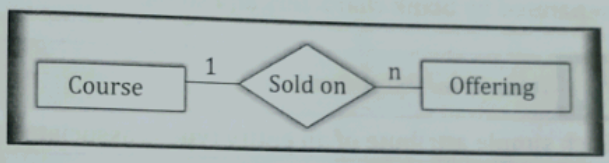
Discriminator can identify a group of entities from the entity set. Discriminator is represented by underlining with a dashed line.

Note:

- The combination of discriminator and primary key of the strong entity set makes it possible to uniquely identify all entities of the weak entity set. Thus, this combination serves as a primary key for the weak entity set

Example

In a university (or) college, two entity sets are Course and Course offering. Suppose an instance of ***Course is "Advanced Database"*** and an instance of Course offering is also **"Advanced Database".**



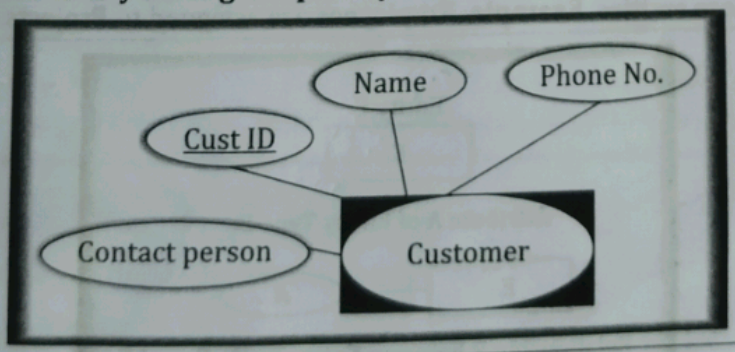
Definition: Weak Entity

Entity types that do not have key attributes of their own are called weak entity type.

2.4.4 Key Attribute of an Entity Type

1. An important feature of an entity type is the ***key* (or) *uniqueness constraint*** on attributes.
2. An entity type might have an attribute whose values are distinct for each individual entry.
3. Such attribute is called *key attribute* and its value can be used to identify each entity uniquely.
4. Sometimes, several attributes together can form a key, meaning that the combination of them must be distinct for each individual entity.
5. Some entity types have more than one key attribute ***for example*** both National Insurance Number and Staff Number are valid keys for the entity type "lecturer".

Example **Customer Entity having unique key attribute Cust_ID**



Definition: Key Attribute

- A key attribute of an entity type is one whose value uniquely identifies an entity of that type

2.4.5 Value Sets (Domain) of Attributes

Domain of attribute specifies the set of values that may be assigned to that attribute for each individual entity.

Example

If the range of ages allowed for employees is between 16 and 70, we can specify the value set of the Age attribute of EMPLOYEE to be the set of integer numbers between 16 and 70. Similarly, we can specify the value set for the Name attribute as being the set of strings of alphabetic characters separated by blank characters and so on.

Definition: Value Sets

Each simple attribute of an entity type is associated with a value sets (or domain of values).

2.5 Relationship

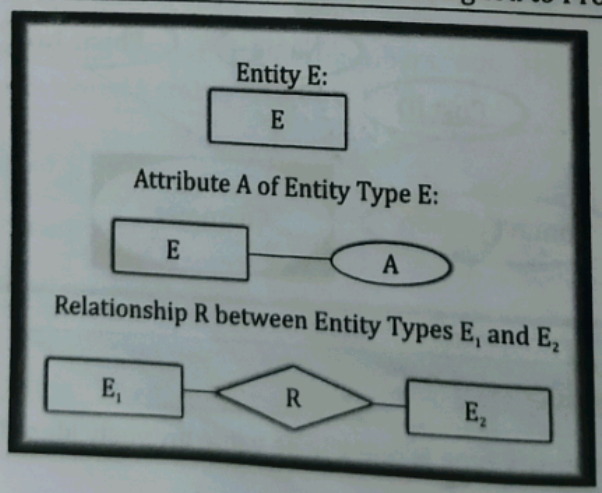
2.5.1 Relationship Definition

A relationship is used to describe the relation between entities. A *relationship* is a meaningful association between entity types. Relationship types are represented on the ER diagram by a series of lines. The relationship is placed inside a diamond,

Example **Managers Manage Employees:**

Definition: Relationship

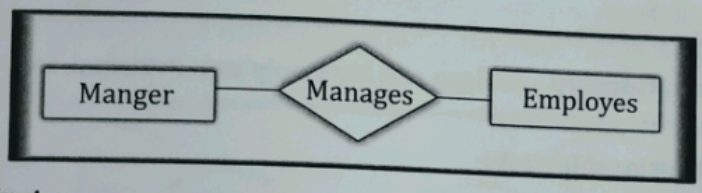
Relationship is an association between entities. Typically, a relationship is indicated by a verb connecting two or more entities. **Example**: Employees are assigned to Projects.



2.5.2 Relationship Types, Sets and Instances

(A) Relationship Type:

A relationship type represents the association between entity types. For example, For example,'Manages' is a relationship type that exists between entity type Manager and Employees. In ER diagram, relationship type is represented by a diamond and connecting the entities with lines.



(B) Relationship Instance:

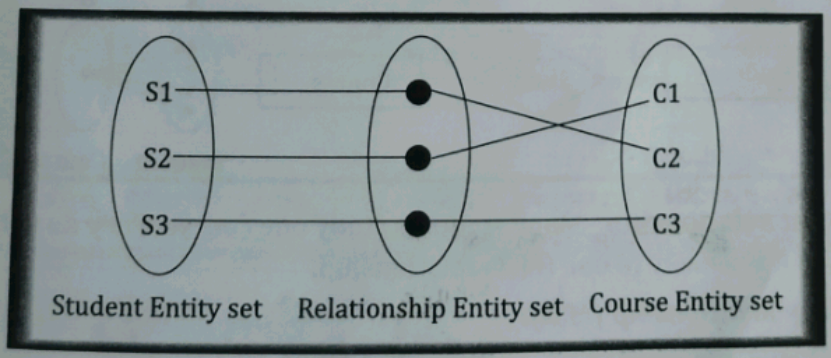
Relationship instance is link between entities. A Relationship Instance is an association of entities, with exactly one entity from each participating entity type.

Example, If 'B' is the relationship between the Employee entity and the Department entity, then Ram works for Comp. Sc department, Shyam works for Electrical department etc. are relationship instances of the relationship, works for.

(C) Relationship Set:

A Relationship set is a set of relationships of the same type. A **relationship set** is a set of "similar" relationships (the similarity is based on the type of underlying entities involved in each such relationship)

Example: The following relationship set depicts S1 is enrolled in C2, S2 is enrolled in C1 and S3 is enrolled in C3.



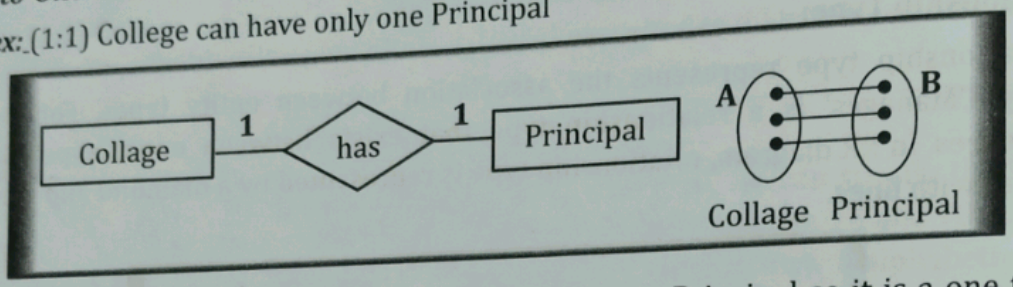
2.5.3 Relationship Between Entity Sets

The Relationship between entity set are of four types

1. One-to-One Relationship (1:1)
2. One-to-Many Relationship (1:N)
3. Many-to-One Relationship (M:1)
4. Many-to-Many Relationship (M:N)

1. ***One-to-One Relationship (1:1):***

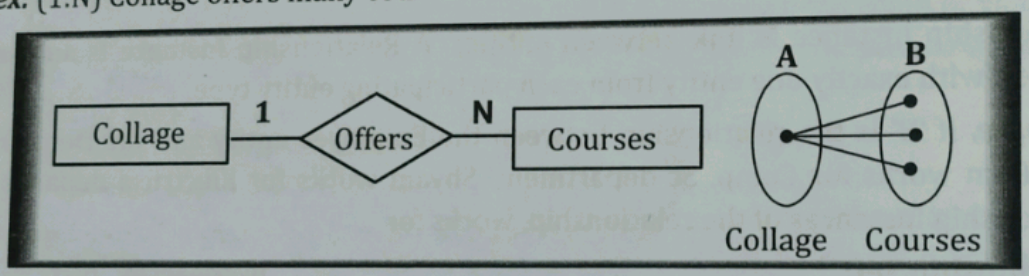
   For ex: (1:1) College can have only one Principal



A one to one relationship - College can have only one Principal so it is a one to one (1:1) relationship.

2. ***One-to-Many Relationship (1:N):***

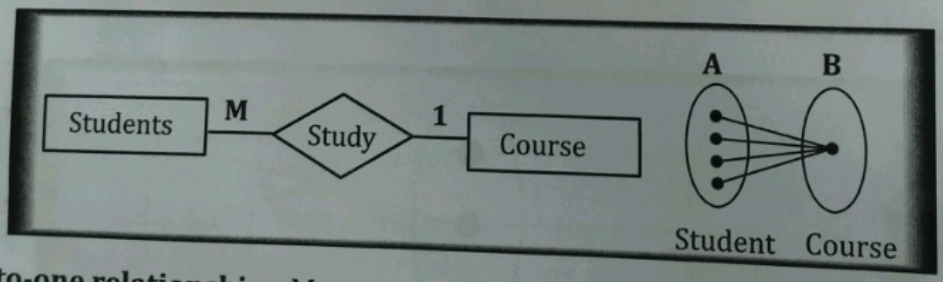
   For ex: (1:N) Collage offers many courses



A one-to-many relationship - one collage can offers many courses like BCA, BBM, B.Com etc so it is a one-to-many (1:N) relationship.

3. ***Many-to-One Relationship (M:1):***

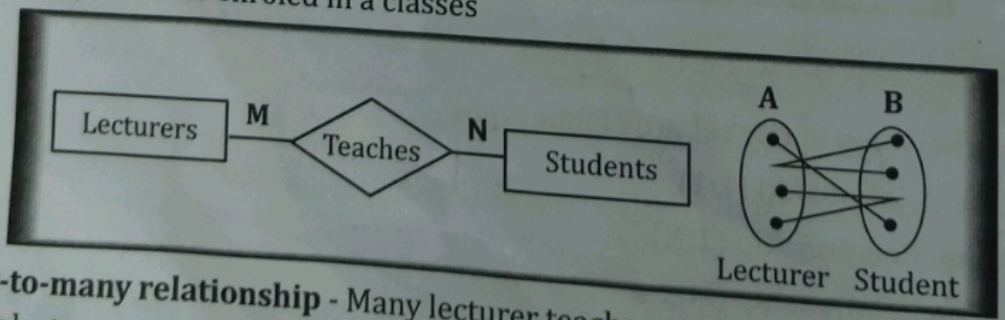
   For ex: (M:1) students study course



A many-to-one relationship - Many students study one course. They do not study more than one course, so it is a many to one (M:1) relationship.

4. ***Many-to-Many Relationship (M:N):***

   For ex: (M:N) students enroled in a classes



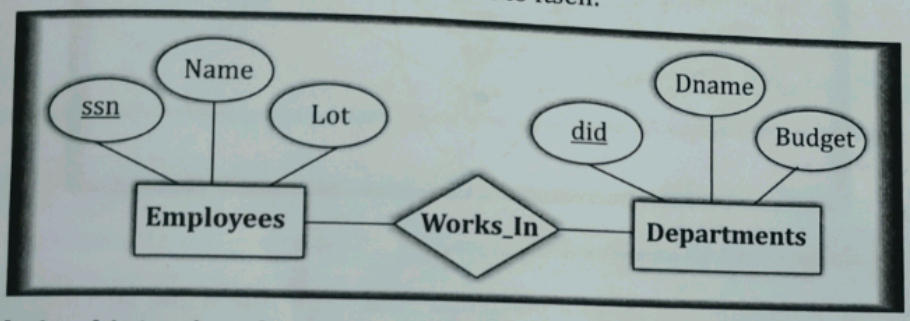
A many-to-many relationship - Many lecturer teaches many students and a student is taught by many lecturers, so it is a many-to-many (M:N) relationship.

2.5.4 Degree of Relationship

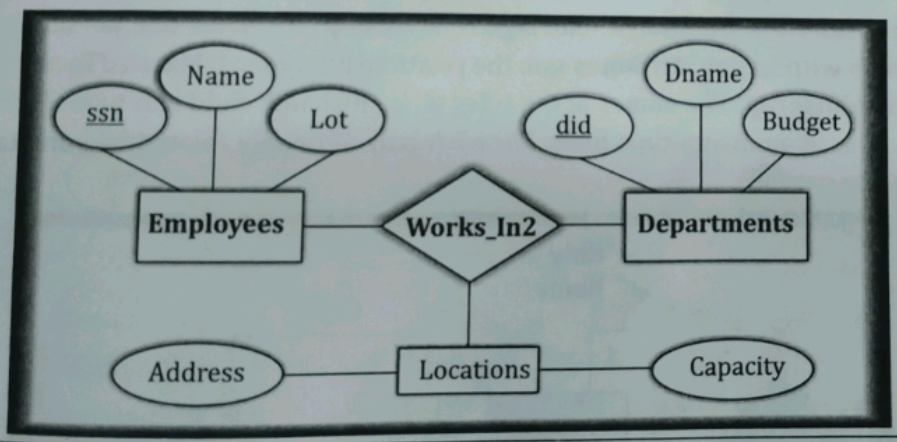
The degree of a relationship type is the number of participating entity types. Hence, the WORKS_ In relationship is of degree two. A relationship type of degree two is called ***binary*** and degree of three is called ***ternary***.

Example

A ***binary relationship*** is a type of relationship between the entities, which links to types of entities in it. It is the association between two entities is the most common type in the real world. A recursive binary relationship occurs when an entity is related to itself.



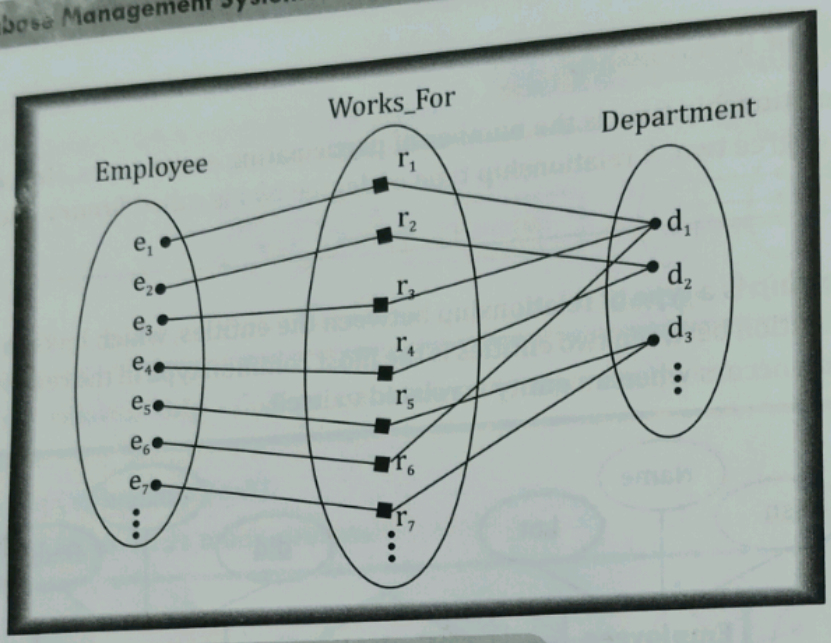
A ***ternary relationship*** involves three entities and is used when a binary relationship is inadequate. Many modeling approaches recognize only binary relationships. Ternary (or) n-ary relationships are decomposed into two or more binary relationships.



2.5.5 Relationship As Attributes

Some instances of WORKS_FOR relationship set, which represents a relationship type WORKS_FOR between EMPLOYEE and DEPARTMENT. Relationship types can have attributes of 1:1 (or) 1:N relationships, attributes can be migrated to one of the participating entity types.

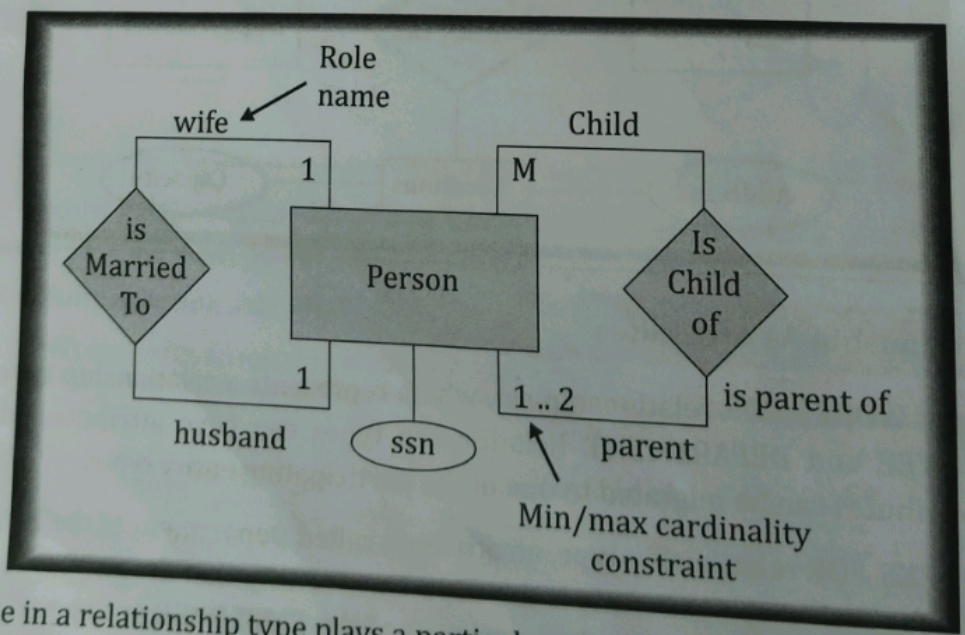
Consider the WORKS_FOR relationship type, an attribute called Department of the EMPLOYEE entity type whose value for each employee entity is the department entity that the employee works for. The value set of this Employees attribute is the power set of the EMPLOYEE entity set. Either of these two attributes are Department of EMPLOYEE (or) Employees of DEPARTMENT can be represent the WORKS_FOR relationship type.



2.5.6 Role Names and Recursive Relationship

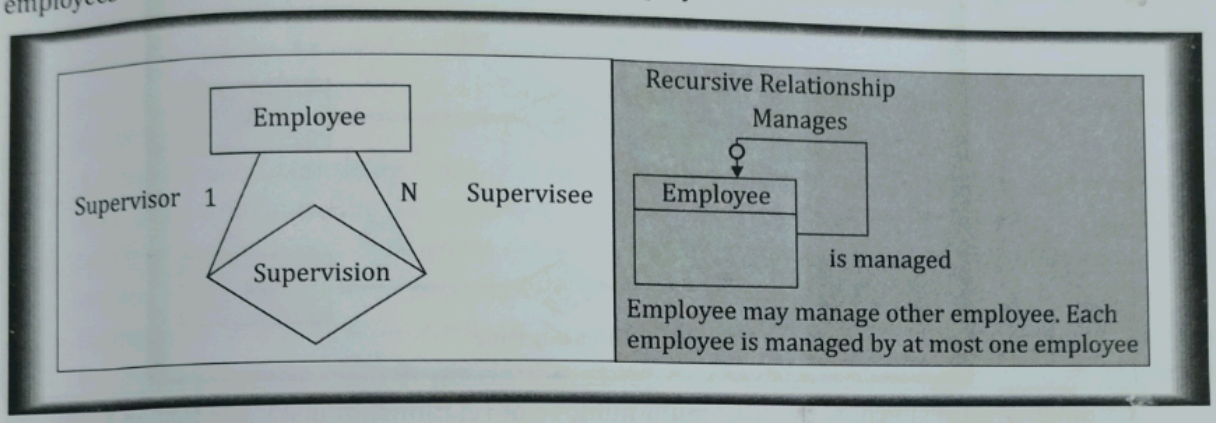
Each entity type in a relationship plays a particular role. The role name specifies the role that a participating entity type plays in the relationship. ***Example,*** in the relationship between Employee and Department, the Employee entity type plays the employee role, and the Department entity type plays the department (or) employer role.

For example, consider the traditional marriage relationship. It relates one person to another. The entity class Person with its key attributes ssn, the relationship types IsMarriedTo and IsChildOf with their lines and cardinalities. The names of the roles wife, husband, child and parent are shown in the diagram next to the relationship-type lines. For each pair of people related by parentage, one is the child and one is the parent.



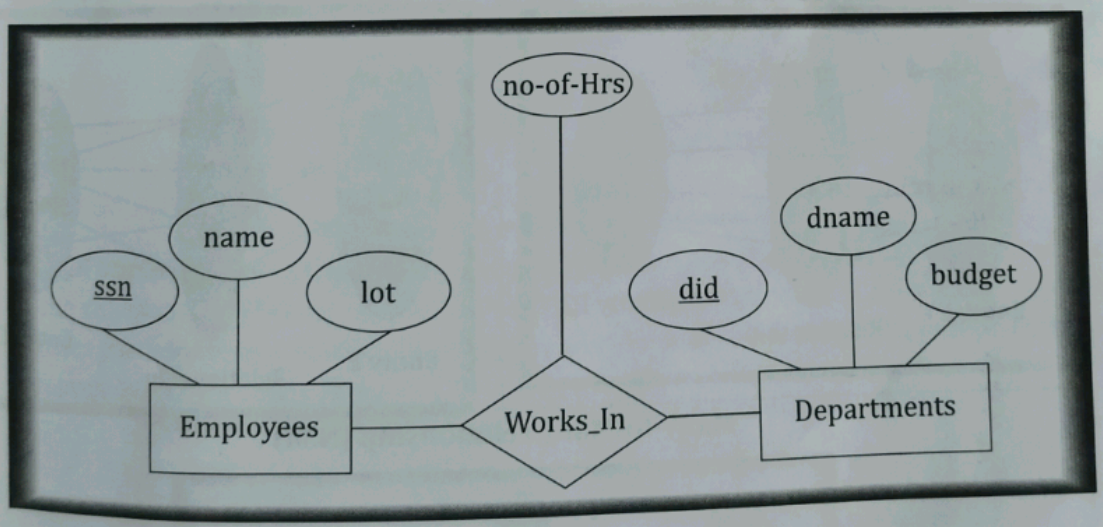
Each entity type in a relationship type plays a particular ***role*** that is described by a ***role name***. Role names are especially important in ***recursive*** relationship types where the same entity participates in more than one role.

A recursive relationship is an entity is associated with itself. An employee may manage many employees and each employee is managed by one employee.



2.5.7 Attributes of Relationship Types

Relationships can have attributes similar to entity types. E***xample,*** in the relationship Works - In between the Employee entity and the Department entity we would like to keep track of the number of hours an employee works on a project. Therefore we can include Number of Hours as an attribute of the relationship.



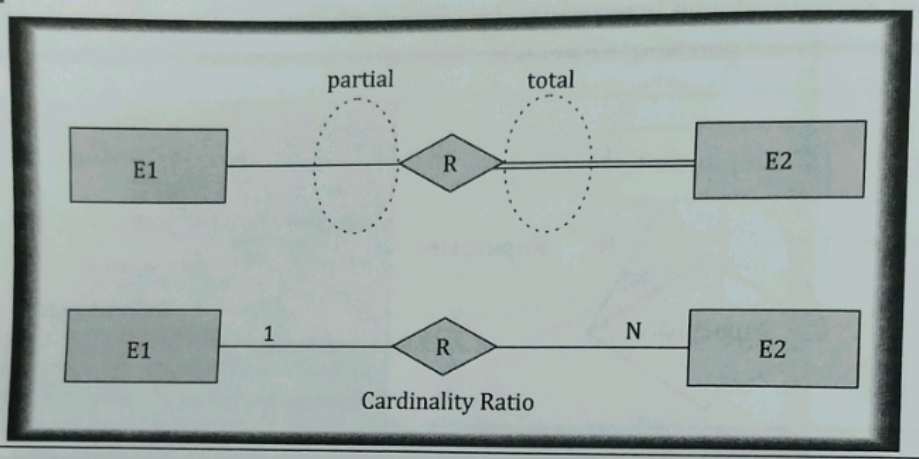
2.5.8 Constraints on Relationship Types

Structural constraints limit the possible combinations of entities that can participate in a relationship instance. ***Structural constraints of a relationship type are:***

1. Cardinality ratio
2. Participation

1. **Cardinality Ratio** specifies the number of relationship instances that an entity can participate in (one-to-one, one-to-many, many-to-many). It limits the number of relationship instances an entity can participate.

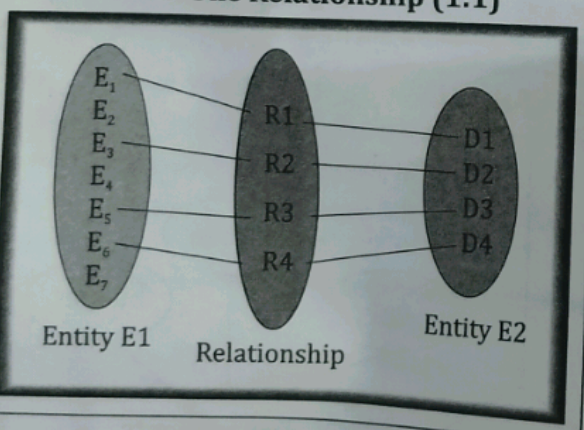
Example



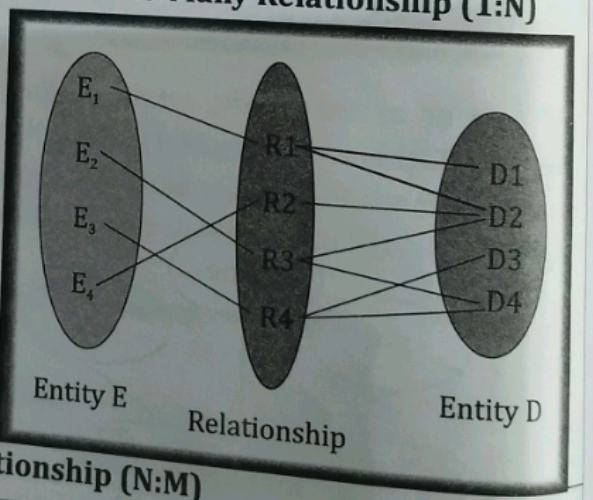
Cardinality ratio specifies the maximum number of relationship instances that an entity can participate

- One-to-One Relationship (1:1)
- One-to-Many relationship (1:N)
- Many-to-Many relationship (N:M)

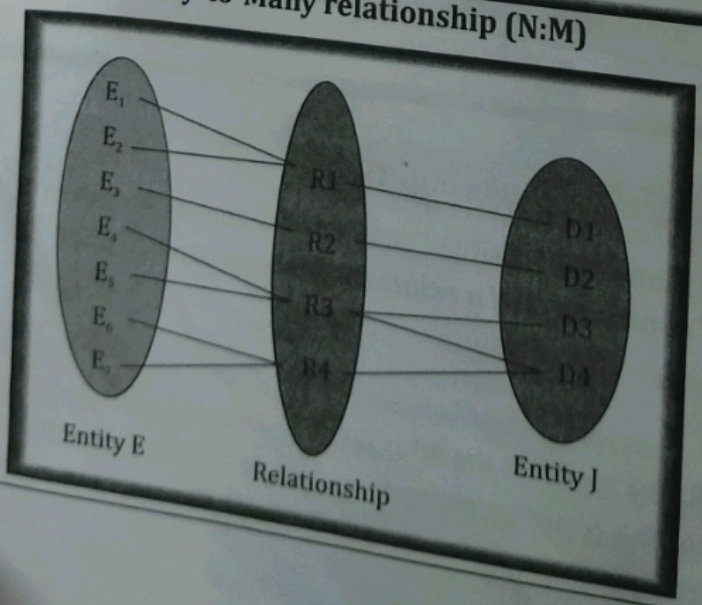
One-to-One Relationship (1:1)



One-to-Many Relationship (1:N)



Many-to-Many relationship (N:M)



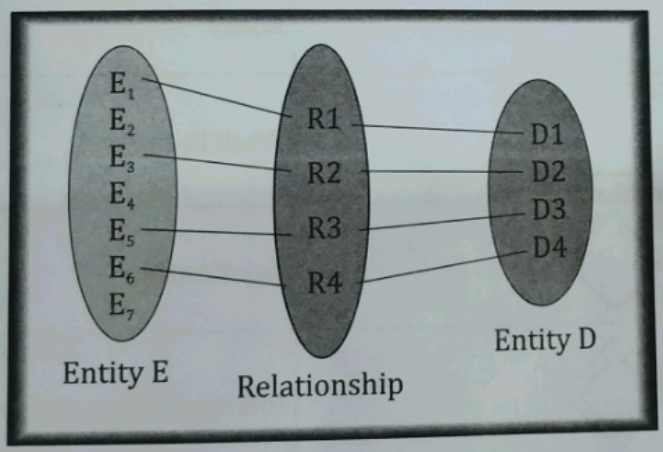
vo Y200 Pro 5G
8 2025 14:31

2. **Participation Constraints** Participation Constraints specifies whether the existence of an entity depends on its being related to another entity via the relationship type. It specifies the minimum number of relationship instance that each entity can participate in and is sometimes called the ***minimum cardinality constraint.*** If each entity of an entity type is **required** to participate in some instance of a relationship type then that participation is **total** otherwise it is **partial.**

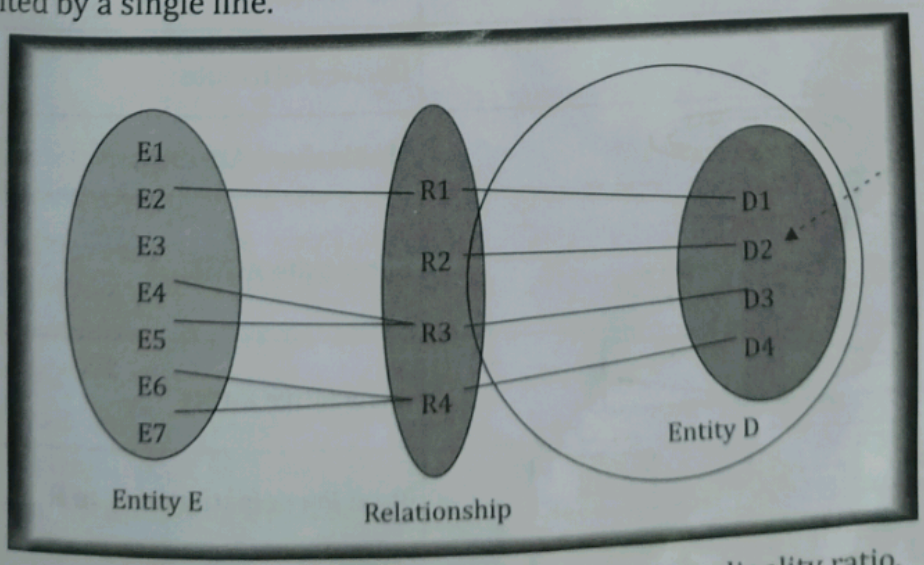
A ***total participation constraint*** (or) existence dependency specifies that an entity can only exist if it participates in the specified relationship (***For Ex:*** every lecturer must work in a specified department). If an entity can exist, only if it participates in at least one relationship instance, then that is called total participation.

Example If company policy states that every employee must work for a department, then an employee can exist only if it participates in at least one relationship instance (i.e. an employee can't exist without a department). It is also sometimes called an ***existence dependency.***

Total participation is represented by a double line, going from the relationship to the dependent entity.



Partial participation constraint specifies there may exist an entity which does not participate in the relationship (***For Ex:*** not all lecturers supervise students). If only a part of the set of entities participate in a relationship, then it is called partial participation partial. Participation is represented by a single line.



Example : Total participation and partial participation in cardinality ratio.

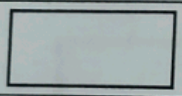
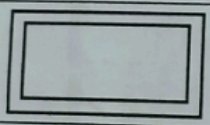
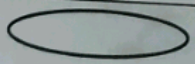
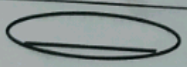
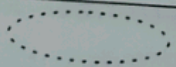
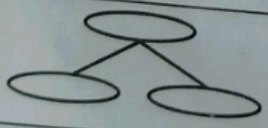
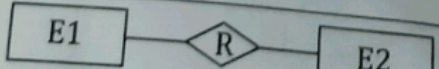
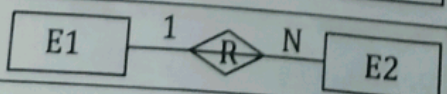
Y200 Pro 5G
25, 14:31

2.6 ER Diagram Notation

The graphical representation of organizational system elements and the association among the elements is called ER Diagram.

- The elements that make up a system are referred to as entities.
- A relationship is the association that describes the interaction between entities.
- An ER diagram may also indicate the cardinality of a relationship.
- The ER notation also allows for distinguishing different types of entities.

The database designer creates an **entity-relationship (E-R) diagram** to show the entities for which information needs to be stored and the relationship between those entities. E-R diagrams use several geometric shapes, but we use only a few of them here: Rectangles represent entity sets, Ellipses represent attributes, Diamonds represent relationship sets, Lines link attributes to entity sets and link entity sets to relationships sets.

Symbol	Meaning
	Entity
	Weak Entity
	Relationship
	Identifying Relationship
	Attribute
	Key Attribute
	Derived Attribute
	Multivalued Attribute
	Composite Attribute
	Associative Entity
	Total Participation of E_2 in R
	Cardinality ratio 1 : N for E_1:E_2 in R

vo Y200 Pro 5G

- A plain rectangle is used for what is termed as ***Fundamental entity*** which is a person, place (or) thing.
- The term ***Associative entity*** is used for some thing that is created that joins two entities.
- Entity types that do not have key attributes of their own are called ***Weak Entity type***.
- ***Attribute*** is a property (or) characteristics of an entity.
- ***Key attribute*** is used to identify an entity uniquely.
- A ***Multivalued attribute*** can have more than one value.
- A ***Derived attribute*** is based on another attribute.
- Relationship specifies how two entities share information in the database.
- Weak entity types are identified by being related to specific entities from another type in combination with one of their attribute value is called ***identifying relationship***.

Example

Symbol	Representation and Example
Entity	Employee
Weak entity	Employee job-history
Relationship	works–in
Attribute	
Identified (key)	emp-id
Descriptor (nonkey)	emp-name
Multivalued descriptor	degrees
Complex attribute	address: street, city, state, zip-code

Attributes are shown in ovals, and each attribute is attached by a straight line to its entity type or relationship type. Component attributes of a ***composite attribute*** are attached to the oval representing the composite attribute, as illustrated by the Name attribute of EMPLOYEE. ***Multivalued attributes*** are shown in double ovals, as illustrated by the Locations attribute of DEPARTMENT. ***Key attributes*** have their identical number like roll_no that names underlined. ***Derived attributes*** are shown in dotted ovals, as illustrated by the Number_Of_Employees attribute of DEPARTMENT. ***Weak entity types*** are distinguished by being placed in double rectangles and by having their identifying relationship placed in double diamonds as illustrated by the DEPENDENT entity type and the DEPENDENTS_OF identifying relationship type. The partial key of the weak entity type is underlined with a dotted line.

Y200 Pro 5G

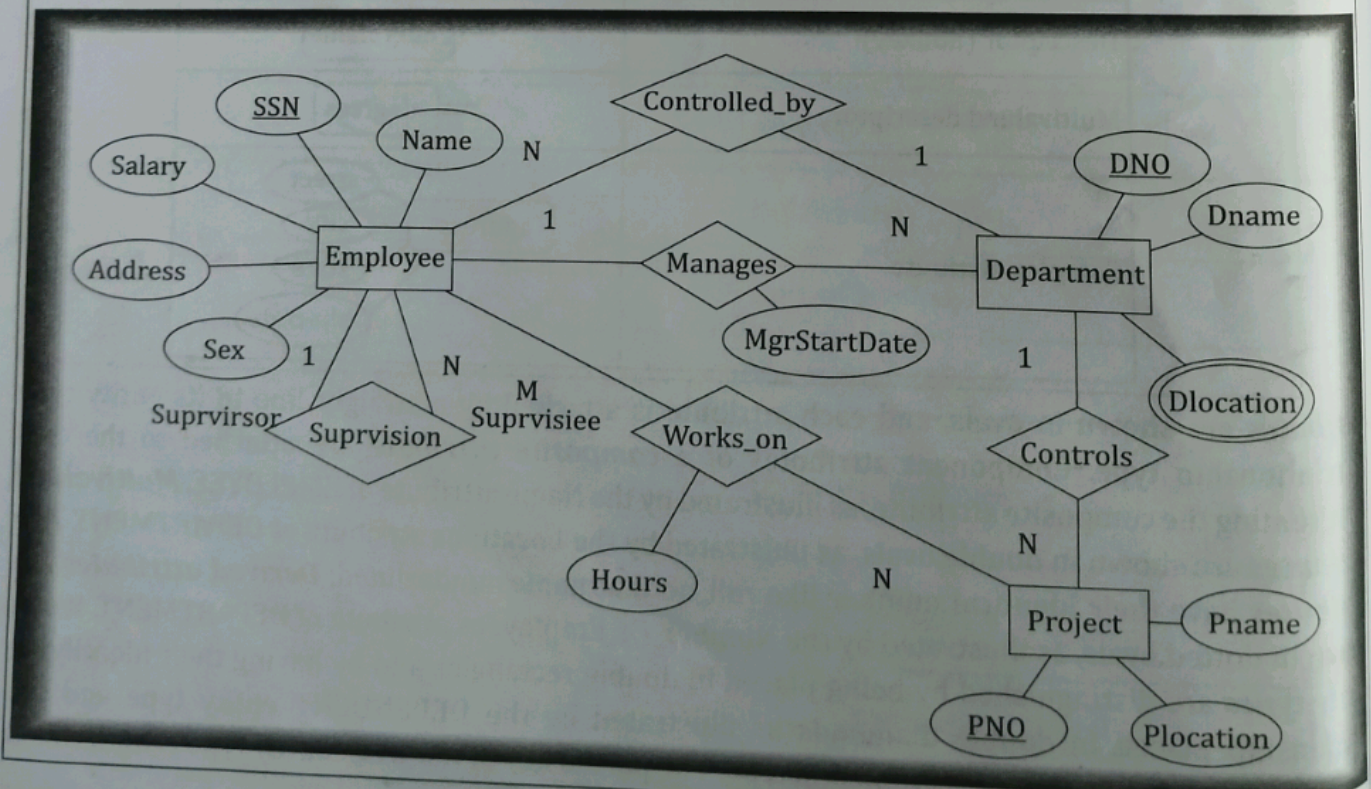
2.6.1 ER-Diagram for Company Database

Example **Example of Database Application**

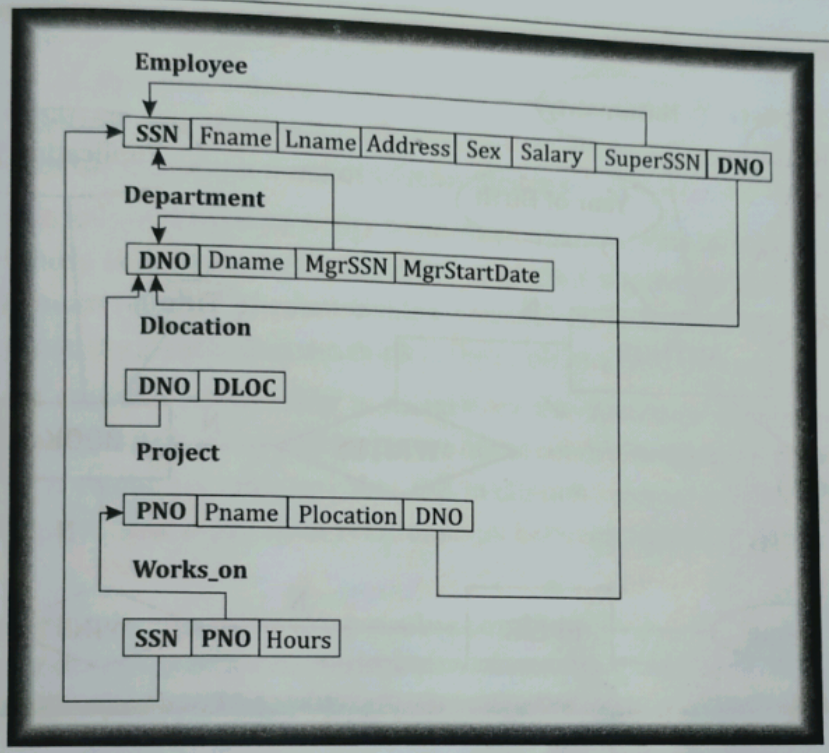
An example database application is COMPANY Database. We list the data requirements for the database here, and then create its conceptual schema step by step.

The COMPANY database keeps track of a company's employees, departments, and projects. Suppose that after the requirements collection and analysis phase, the database designers provided the following description of the "miniworld"- the part of the company to be represented in the database:

- The company is organized into departments. Each department has a unique name, a unique number and a particular employee who manages the department. We keep track of the start date when that employee began managing the department. A department may have several locations.
- A department controls a number of projects, each of which has a unique name, a unique number and a single location.
- We store each employee's name, social security number, address, salary, sex, and birth date. An employee is assigned to one department but may work on several projects, which are not necessarily controlled by the same department. We keep track of the number of hours per week that an employee works on each project. We also keep track of the direct supervisor of each employee.
- We want to keep track of the dependents of each employee for insurance purposes. We keep each dependent's first name, sex, birth date and relationship to the employee.



vivo Y200 Pro 5G
ep 8, 2025, 14:31

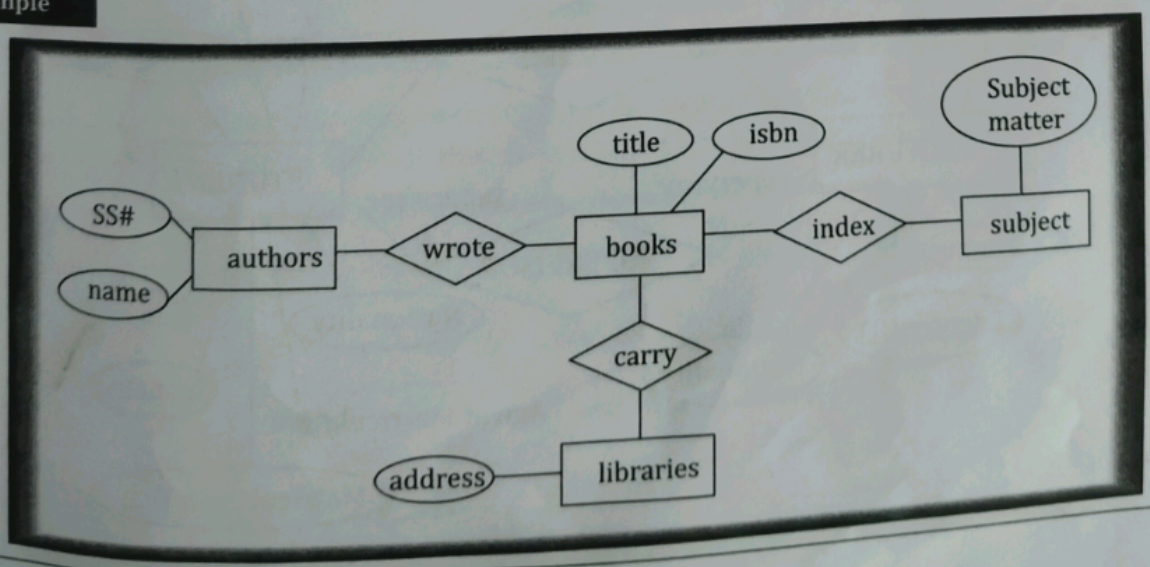


Schema Diagram

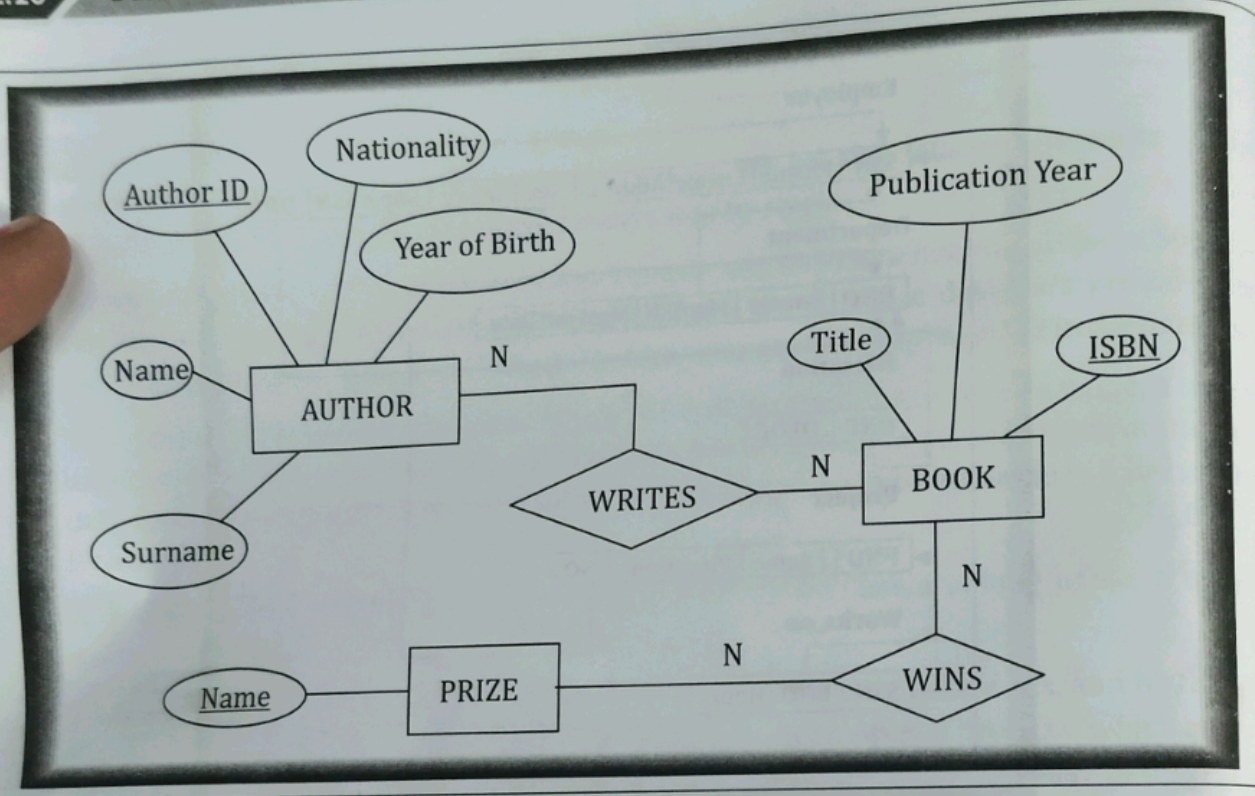
2.6.2 Proper Naming of Schema Constructs

When designing a database schema, the choice of names for entity types, attributes, relationship types. In ER diagrams use the convention that entity type and relationship type names are in upper case letters, attribute names are capitalized and role names are in lower case letters. A narrative description of the database requirements, the nouns appearing in the narrative tend to give rise to entity type names and the verbs tend to indicate names of relationship types. Attribute names generally arise from additional nouns that describe the nouns corresponding to entity types.

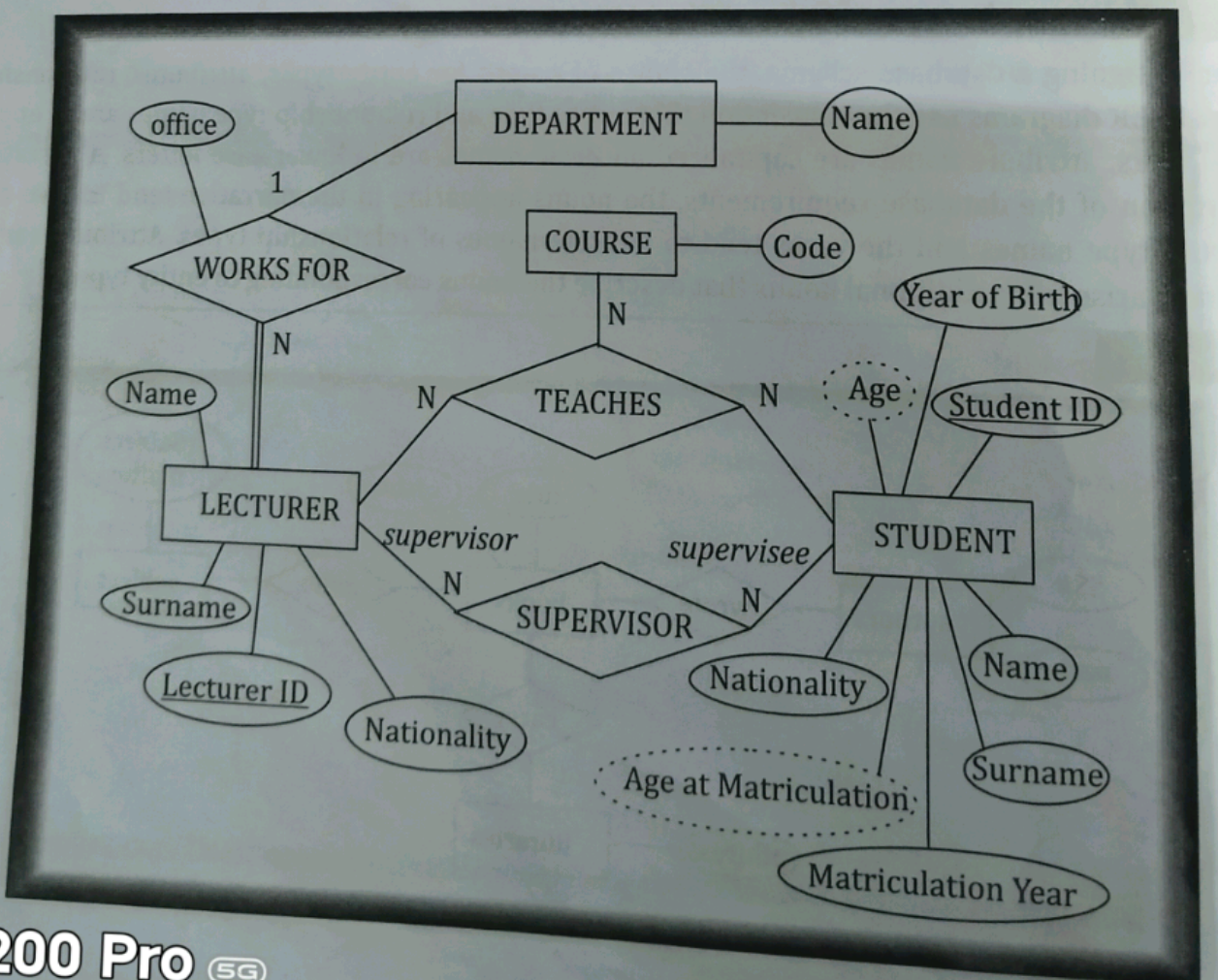
Example



vo Y200 Pro 5G



2.6.3 ER Diagram for University



VO Y200 Pro 5G
8, 2025, 14:31

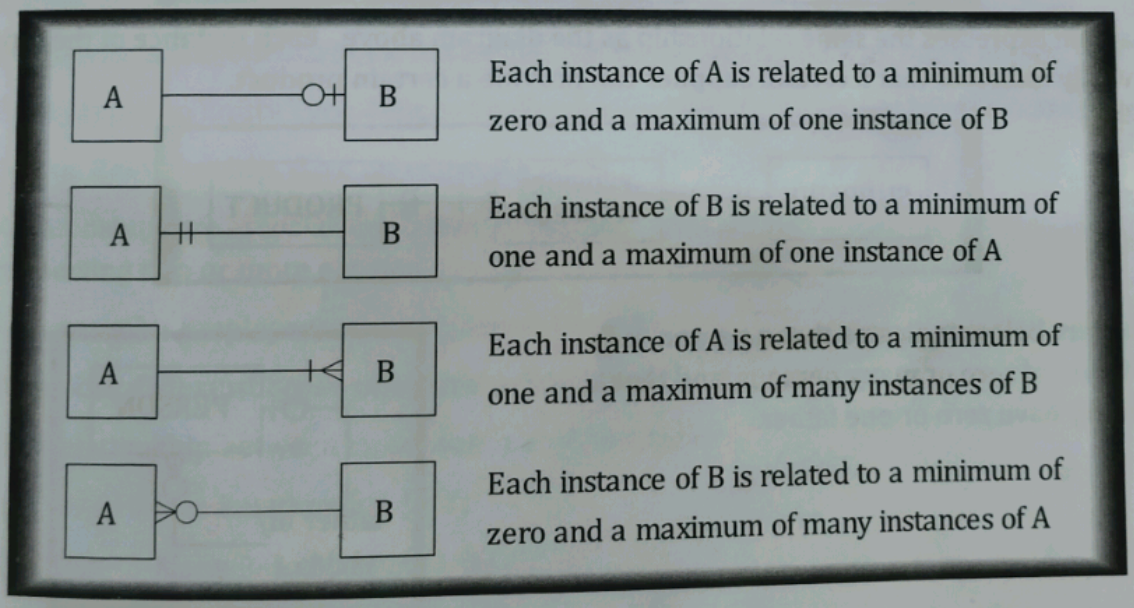
2.6.4 Optionality and Cardinality

Symbols at the ends of the relationship lines indicate the optionality and the cardinality of each relationship. ***"Optionality"*** expresses whether the relationship is optional or mandatory. ***"Cardinality"*** expresses the maximum number of relationships.

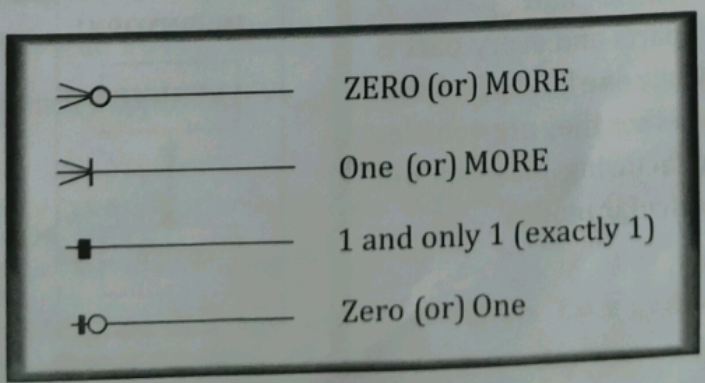
As a relationship line is followed from an entity to another, near the related entity two symbols will appear. The first of those is the optionality indicator. A ***circle (O)*** indicates that the relationship is optional—the minimum number of relationships between each instance of the first entity and instances of the related entity is zero. One can think of the circle as a zero, (or) a **letter O** for "optional."

A ***stroke (|)*** indicates that the relationship is mandatory the minimum number of relationships between each instance of the first entity and instances of the related entity is one. The second symbol indicates cardinality. A stroke (|) indicates that the maximum number of relationships is one. A ***"crows-fooi"*** (⋞) indicates that many such relationships between instances of the related entities might exist.

The following diagram indicates all of the possible combinations:

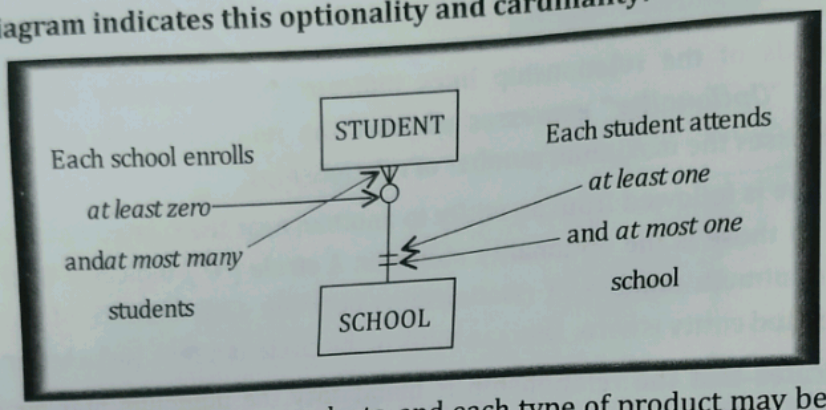


In our model, we wish to indicate that each school may enroll many students, or may not enroll any students at all. We also wish to indicate that each student attends exactly one school.

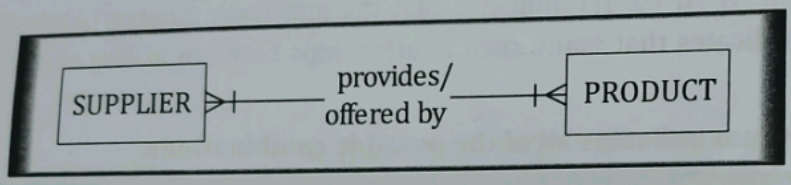


Y200 Pro

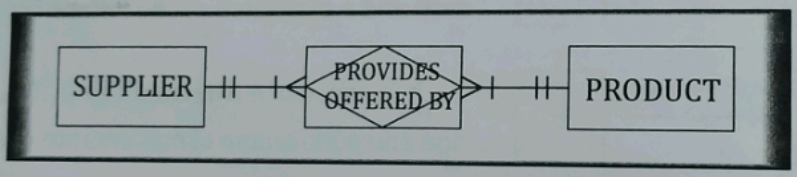
The following diagram indicates this optionality and cardinality:



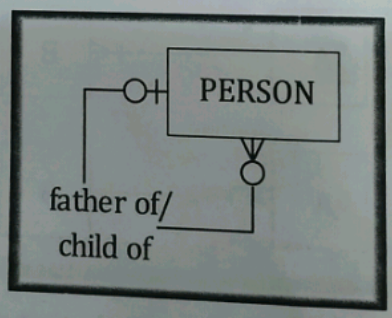
A supplier may provide many different products, and each type of product may be offered by many suppliers:



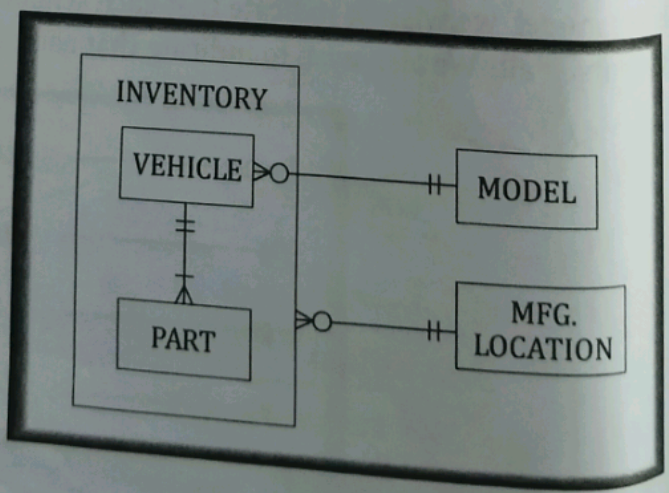
This diagram expresses the same relationship as the diagram above. Each instance of the "provides" bridge entity indicates that a certain supplier can provide a certain product.



The diagram below indicates that a person may be the father of zero or many persons, and that a person may have zero or one father.

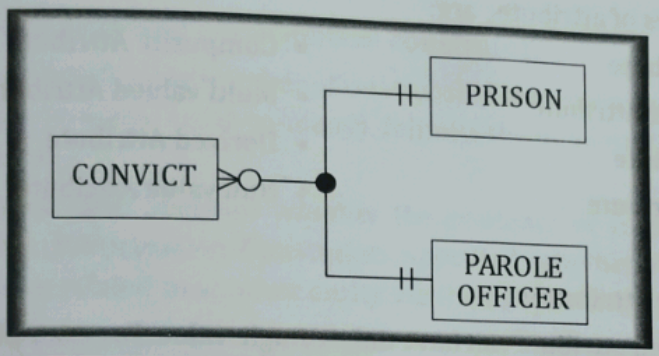


The example above depicts an "inventory" entity, with the subtypes of "vehicle" and "part." A vehicle has one or many parts and every part is associated with one and only one kind of vehicle. All items in inventory, whether they are vehicles (or) parts, have a manufacturing location, but only vehicles are of a particular model.



VO Y200 Pro 5G

The following diagram indicates that each convict is assigned to a prison (or) to a parole officer, but not both.



2.7 Review Question

I. Two Mark Questions And Answers

1. What is an Entity and Entity Type?

Ans. **Entity:** An entity may be any object, class, person or place. (or) Entity is an object in the real world with an independent existence and can be differentiated with other object.

Entity type: An Entity Type defines a collection (or) set of entities that have the same attributes.

2. Define Relationship. Give an example

Ans. Relationships are associations between entities. Typically, a relationship is indicated by a verb connecting two or more entities.

For example: Employees are assigned to Projects

3. Name the different types of relationship types?

Ans. The Relationship between entity sets are of four types:

(a) One-to-One Relationship (1:1)

(b) One-to-Many Relationship (1:N)

(c) Many-to-One Relationship (N:1)

(d) Many-to-Many Relationship (N: N)

4. What is weak entity and strong entity?

Ans. **Weak Entity:** The entity types that do not have key attributes of their own are called weak entity type.

Strong entity: Entity types that have primary keys are called strong entity type.

5. What is an attribute? Give an example.

Ans. An attribute is a property or description of an entity. For Ex: a person has an age, a car has a color.

6. Name different types of attribute.

Ans. The different types of attributes are:

- Simple Attribute
- Single valued Attribute
- Stored Attribute
- Complex Attribute
- Composite Attribute
- Multi valued Attribute
- Derived Attribute
- Null value Attribute

7. Define.

Ans. (a) Single value attributes

A single valued attribute can have only a single value. For example a person can have only one 'date of birth', 'age' etc. A single valued attributes can have only single value. But it can be simple or composite attribute. Date of birth is a composite attribute; 'age' is a simple attribute. But both are single valued attributes.

(b) Composite attribute

A composite attribute is an attribute that can be further subdivided and attribute is value not atomic. For example the attribute ADDRESS can be subdivided into street, city, state, and zip code.

(c) Simple attribute

Simple attribute that consist of a single atomic value. A simple attribute cannot be subdivided. For example the attributes age, sex etc.

(d) Multi-valued attribute

Multivalued attributes can have multiple values. For example a person may have multiple phone numbers, multiple degrees etc. Multivalued attributes are shown by a double line connecting to the entity in the ER diagram.

(e) Stored attribute

Stored attribute that supplies a value to the related attribute. For example 'Date of birth' of a person is a stored attribute.

(f) Derived attribute

The value for the derived attribute is derived from the stored attribute. For example 'Date of birth' of a person is a stored attribute. The value for the attribute 'AGE' can be derived by subtracting the 'Date of Birth'(DOB) from the current date.

(g) Null attribute values

A particular entity may not have an applicable value for an attribute. For example, the apartment_number attribute of an address applies only to addresses that are in apartment buildings and not to other types of residences, such as single-family homes.

8. What is an entity set and key attribute of an entity type?

Ans. Entity set: An Entity Set is a collection of related entities all of the same type. An Entity Set is also known as the extension of an entity type.

Key attribute of an entity type: A key attribute of an entity type is one whose value uniquely identifies an entity of that type.

9. **What is cardinality ratio and participation constraint?**

Ans. **Cardinality ratio:** Specifies the number of relationship instances that an entity can participate in (one-to-one, one-to-many, many-to-many). It limits the number of relationship instances an entity can participate.

Participation constraint: Specifies whether the existence of an entity depends on it being in the relationship. Participation Constraints specified whether the existence of an entity depends on its being related to another entity via the relationship type.

10. **What is generalization and specialization?**

Ans. **Generalization:** Generalization proceeds from the recognition that a number of entity sets share some common features are described by the same attributes and participate in the same relationship sets.

Specialization: The process of designating sub-groupings within an entity set is called specialization.

11. **Define aggregation and ER-diagram?**

Ans. **Aggregation:** Aggregation is an abstraction in which relationship sets (along with their associated entity sets) are treated as higher-level entity sets, and can participate in relationships.

ER-diagram: The graphical Representation of organizational system elements and the association among the elements is called ER Diagram.

12. **What is degree of relationship and binary relationship?**

Ans. **Degree of relationship:** The number of participating entity type is called degree of relationship.

Binary relationship: A Binary Relationship is a type of relationship between entities, which links two types of entities in it.

II. Five Mark Questions

1. Explain tuple, domain, attribute ,entity, relationship with an example.
2. Define relationship? Explain the different types of relationships with example.
3. Describe the different notations used in ER-diagram.
4. What is an attribute? Explain different types of attribute? With an example
5. Define relationship? Explain the different types of relationships with example.
6. Write a short notes on Aggregation and Abstraction

III. Eight Mark Questions

1. Construct an ER-diagram for university database.
2. Construct an ER-diagram for student database.

3. Construct an ER-diagram for company database.
4. Construct an ER diagram for a supermarket data base.
5. Construct an ER diagram for Bank data base.

****